

Reinforcement Learning

Qian Wang

School of Biomedical Engineering
ShanghaiTech University

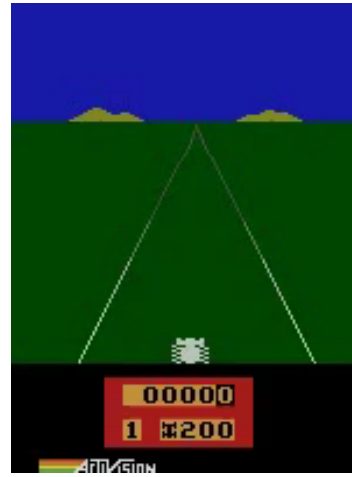
Deep Reinforcement Learning

2013

Atari (DQN)
[Deepmind]



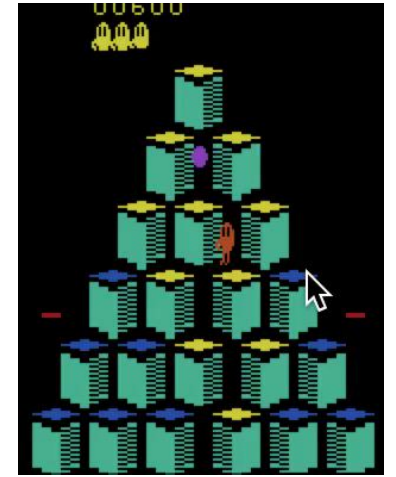
Pong



Enduro



Beamrider



Q*bert

Deep Reinforcement Learning

2013 Atari (DQN)
 [Deepmind]

2015 AlphaGo
 [Deepmind]



Deep Reinforcement Learning

2013	Atari (DQN) [Deepmind]
2015	AlphaGo [Deepmind]
2016	3D locomotion (TRPO+GAE) [Berkeley]



Deep Reinforcement Learning

2013

Atari (DQN)
[Deepmind]

2015

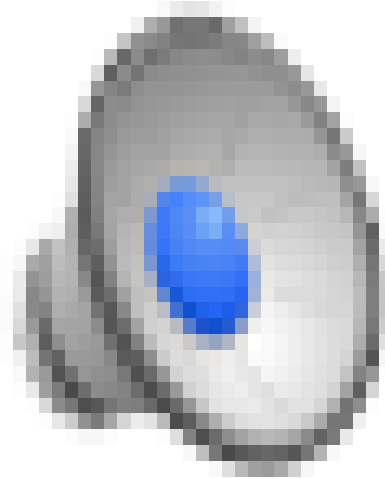
AlphaGo
[Deepmind]

2016

3D locomotion (TRPO+GAE)
[Berkeley]

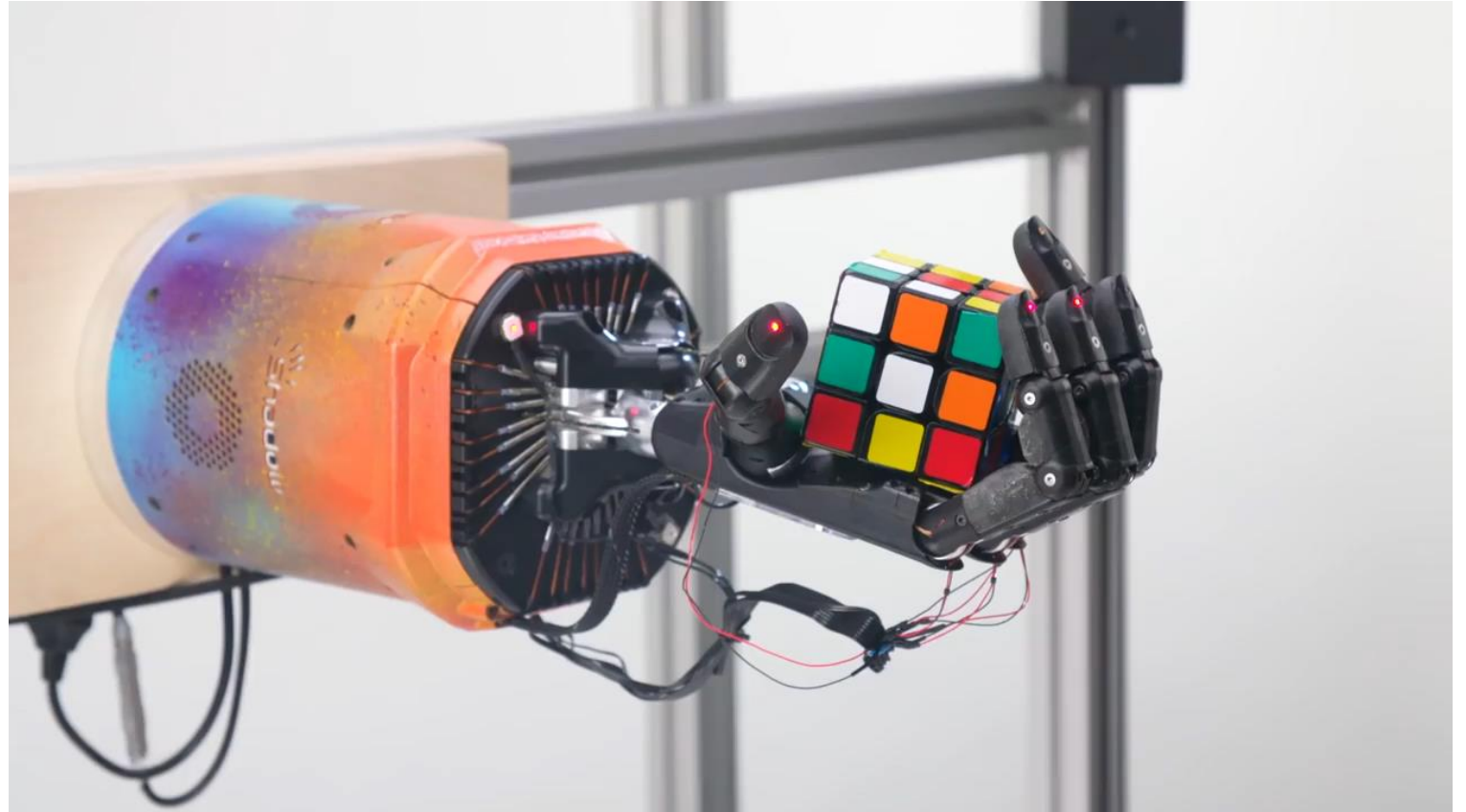
2016

Real Robot Manipulation (GPS)
[Berkeley]



Deep Reinforcement Learning

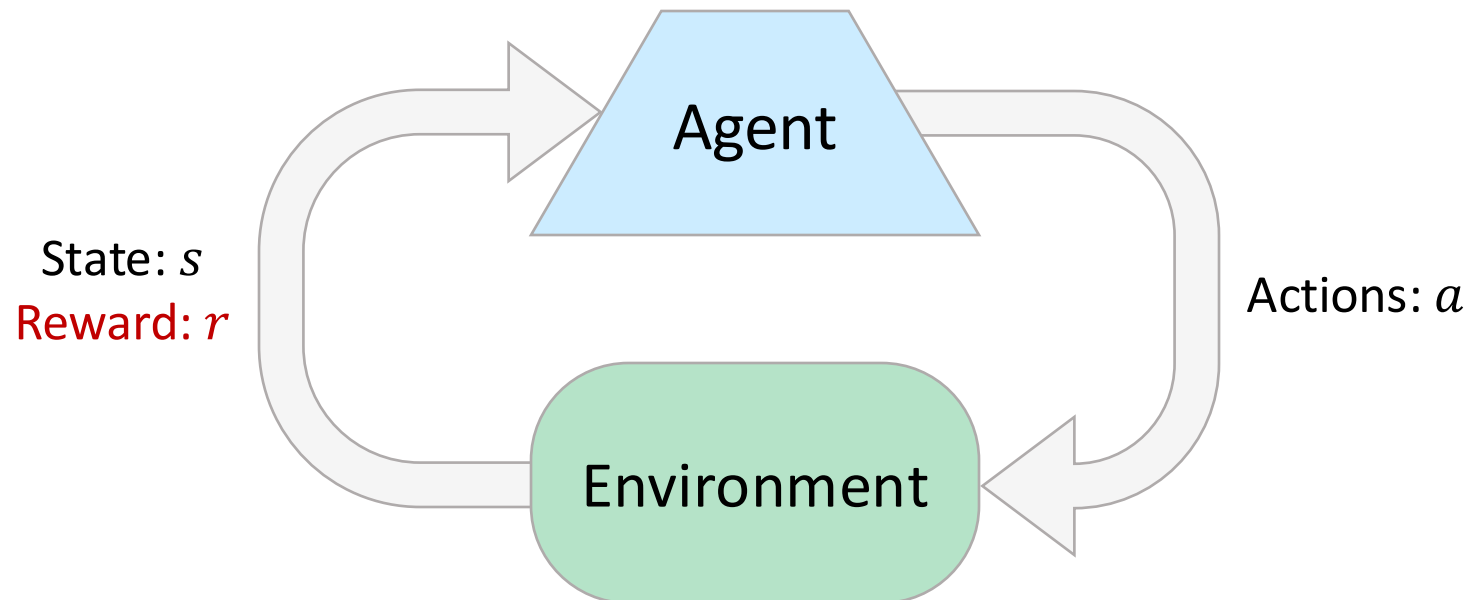
2013	Atari (DQN) [Deepmind]
2015	AlphaGo [Deepmind]
2016	3D locomotion (TRPO+GAE) [Berkeley]
2016	Real Robot Manipulation (GPS) [Berkeley]
2019	Rubik's Cube (PPO+DR) [OpenAI]



OpenAI

Reinforcement Learning

- Basic idea:
 - Receive feedback in the form of **rewards**.
 - Agent's utility is defined by the reward function.
 - Must (learn to) act so as to **maximize expected rewards**.
 - All learning is based on observed samples of outcomes!

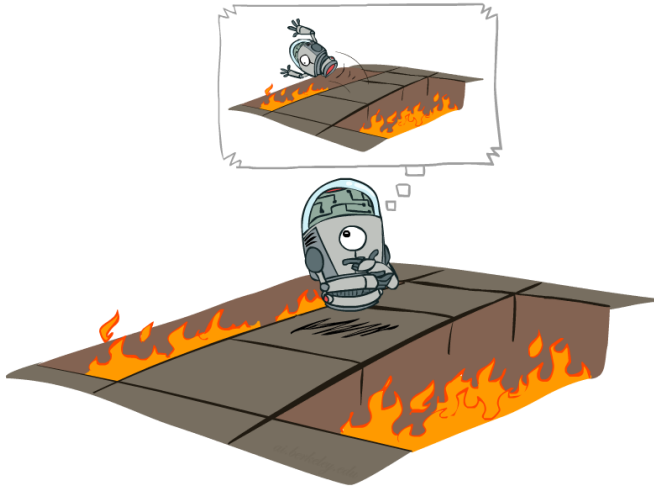


Reinforcement Learning

- Still assume a Markov decision process (MDP):
 - A set of states $s \in S$;
 - A set of actions (per state) A ;
 - A transition model $T(s, a, s')$;
 - A reward function $R(s, a, s')$;
- Still looking for a policy $\pi(s)$.
- New twist: **don't know T or R**
 - We don't know which states are good or what the actions do.
 - Must actually try out actions and states to learn.



Offline (MDPs) vs. Online (RL)



Offline Solution

- Assumes known transition and reward models
- Enables optimal policy computation before interaction
- Suitable for planning-based approaches
- No need for trial-and-error learning

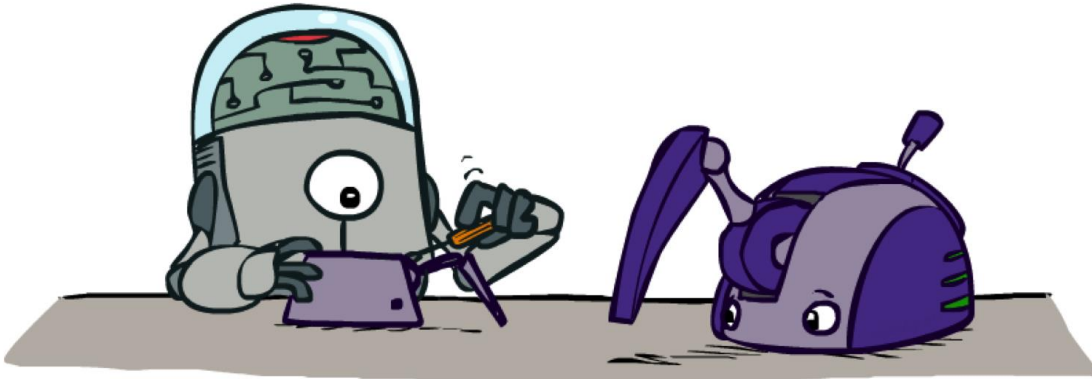


Online Learning

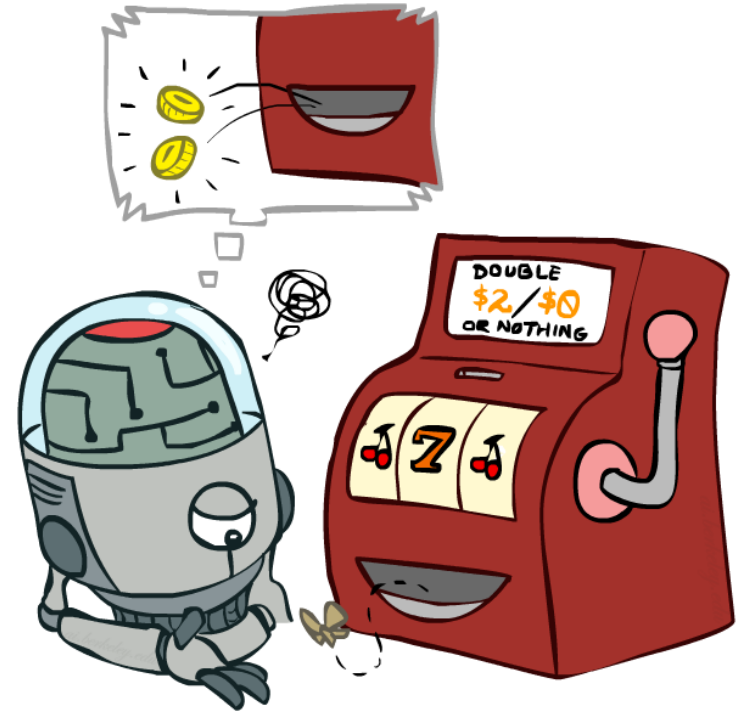
- No prior knowledge of transition or reward models
- Learns optimal policies through interaction
- Relies on trial-and-error experience
- Suitable for dynamic or unknown environments

Model-Based and Model-Free Learning

Model-Based Method

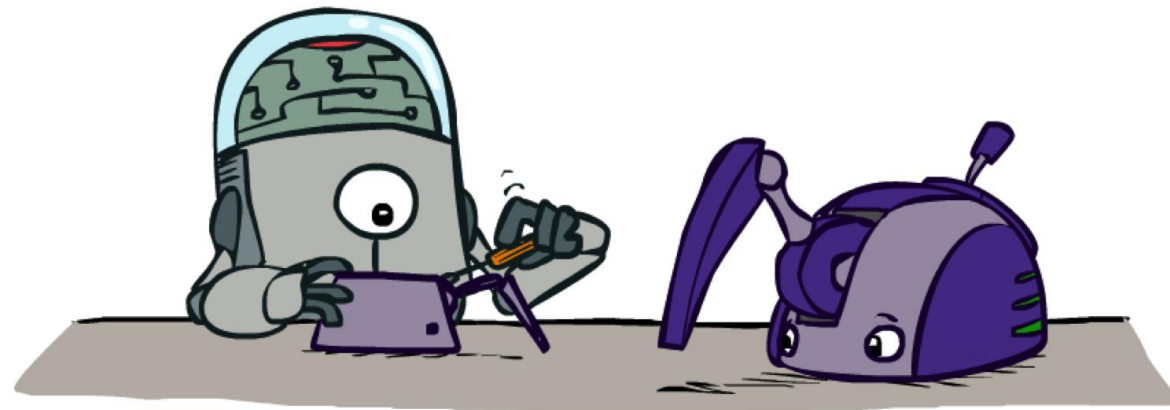


Model-Free Method



Model-Based Learning

- The agent uses a transition model to help interpret the reward signals and to make decisions about how to act.
- Learn the model dynamics from data and then using these models to plan and make decisions.

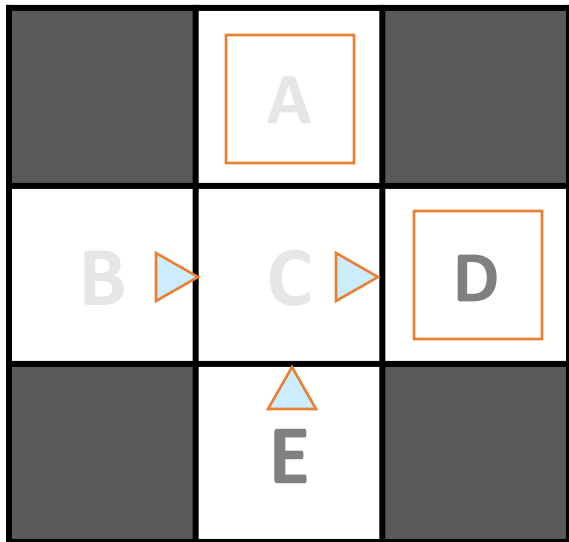


Model-Based Learning

- Idea in overall: Learn an approximate model from experience, and plan as if the learned model were accurate.
 - Step 1: Learn the empirical MDP model
 - Count outcomes s' for each (s, a) ;
 - Normalize counts to give an estimate of $\hat{T}(s, a, s')$;
 - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s') .
 - Step 2: Solve the learned MDP.
 - Apply standard planning algorithms (e.g., value iteration);
 - Compute utility values and derive a policy based on the learned model.

Example: Model-Based Learning

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$$\hat{T}(s, a, s')$$

$T(B, \text{east}, C) = 1.00$
 $T(C, \text{east}, D) = 0.75$
 $T(C, \text{east}, A) = 0.25$
...

$$\hat{R}(s, a, s')$$

$R(B, \text{east}, C) = -1$
 $R(C, \text{east}, D) = -1$
 $R(D, \text{exit}, x) = +10$
...

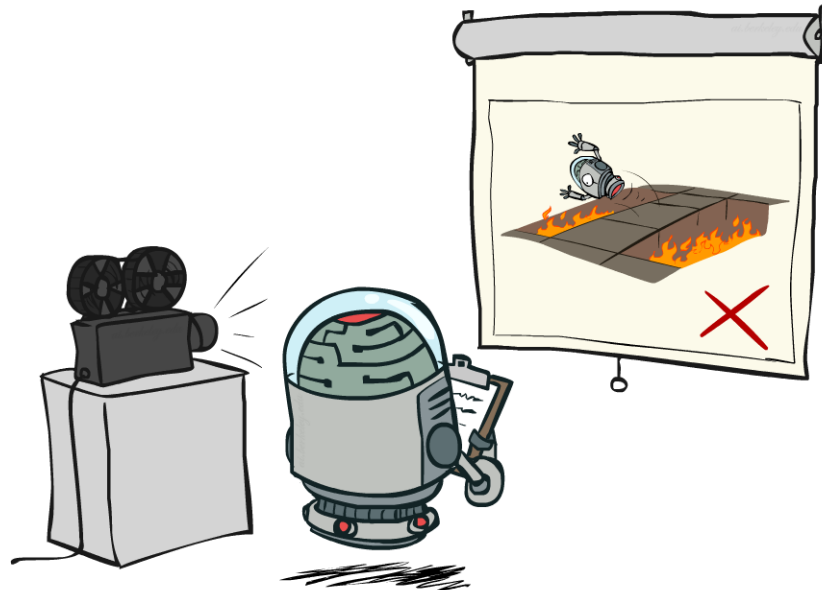
Model-Free Learning

- No transition model is known or learned;
- Learns behavior directly from experience.



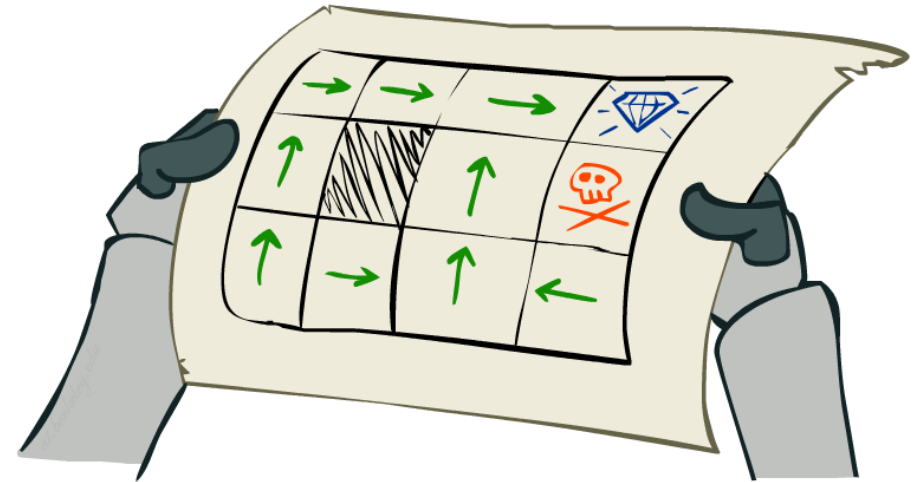
Passive Reinforcement Learning

- We start with an agent that already has a fixed policy $\pi(s)$ to determine its actions.
 - The agent learns from the environment based on that fixed policy without actively trying to improve it.



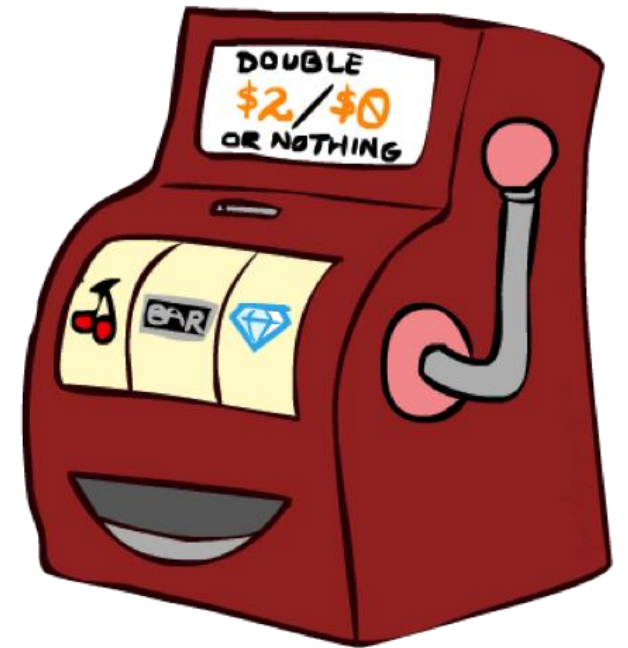
Passive Reinforcement Learning

- Simplified task: policy evaluation
 - Input: a fixed policy $\pi(s)$;
 - You don't know the transitions $T(s, a, s')$;
 - You don't know the rewards $R(s, a, s')$;
 - Goal: learn the state values under π .



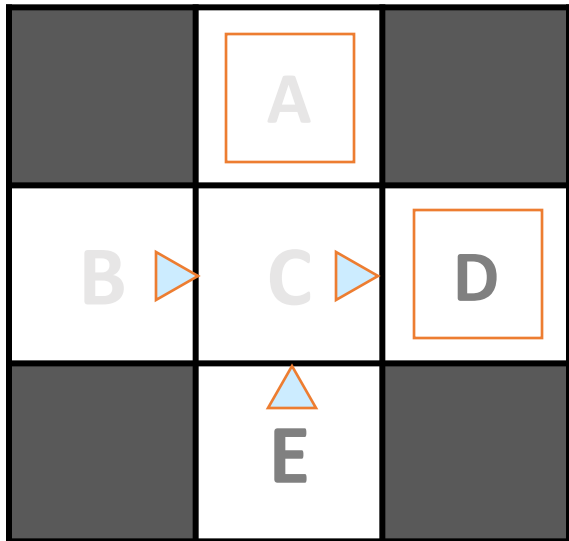
Direct Utility Evaluation

- **Direct evaluation:** Estimates state values without needing a model of the environment.
 - Follow a fixed policy π to generate sample episodes;
 - Record the total (discounted) reward each time a state is visited;
 - Compute the average return for each state across all samples.



Example: Direct Evaluation

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

	-10	
	A	
+8	+4	+10
B	C	D
	-2	
	E	

Problems with Direct Evaluation

- What's good about direct evaluation?
 - Simple and intuitive;
 - No need for transition or reward models;
 - Accurately estimates state values using sample episodes.
- What bad about it?
 - Ignores connections between states;
 - Learns each state independently;
 - Slow convergence due to inefficient use of data.

Output Values

	-10 A	
+8 B	+4 C	+10 D
	-2 E	

If B and E both go to C under this policy, how can their values be different?

Why Not Use Iterative Policy Evaluation?

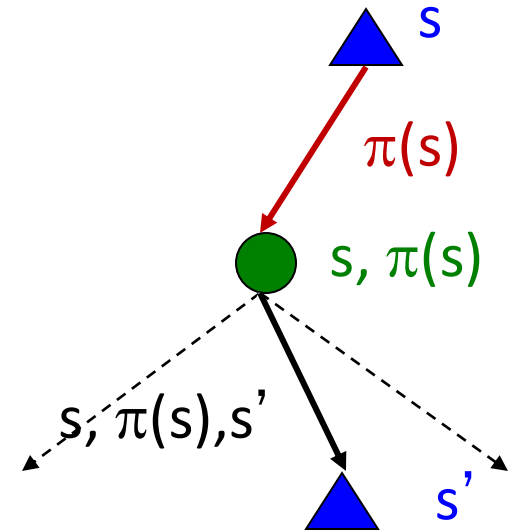
- Simplified Bellman updates calculate V for a fixed policy:

- Each round, replace V with a one-step-look-ahead layer over V

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- This approach fully exploits the connections between the states.
- Unfortunately, we need T and R to do it (yet we don't have)!
- Key question: how can we do this update to V without knowing T and R ?
 - In other words, how to take a weighted average without knowing the weights?



Sample-Based Policy Evaluation

- We want to improve our estimate of V by computing these averages:

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^{\pi}(s')]$$

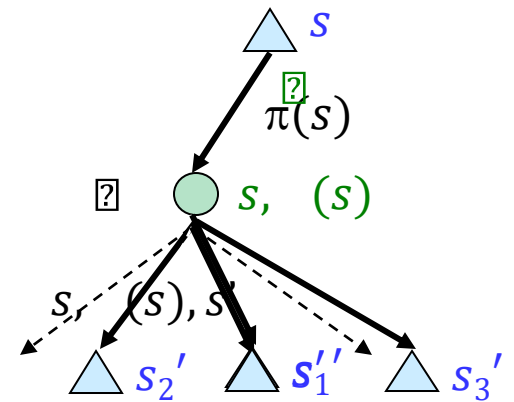
- Idea: Take sample outcomes from the next states (i.e., by doing the action) and then average.

$$sample_1 = R(s, \pi(s), s'_1) + \gamma V_k^{\pi}(s'_1)$$

$$sample_2 = R(s, \pi(s), s'_2) + \gamma V_k^{\pi}(s'_2)$$

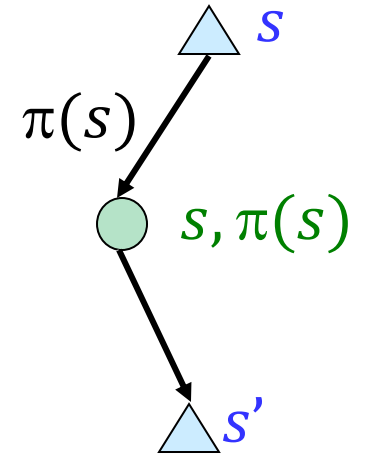
$$sample_n = R(s, \pi(s), s'_n) + \gamma V_k^{\pi}(s'_n)$$

$$V_{k+1}^{\pi}(s) \leftarrow \frac{1}{n} \sum_i sample_i$$



Temporal Difference (TD) Learning

- TD learning: Learn from every experience, not just full episodes!
 - Update value estimates using the observed reward and the estimated value of the next state;
 - Each time a transition (s, a, s', r) occurs, update $V(s)$;
 - Frequent outcomes s' contribute more updates, improving learning efficiency.



Sample of $V(s)$: $sample = R(s, \pi(s), s') + \gamma V^\pi(s')$

Update to $V(s)$: $V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + (\alpha)sample$

Same update: $V^\pi(s) \leftarrow V^\pi(s) + \alpha(sample - V^\pi(s))$

Example: Temporal Difference Learning

States

	A	
B	C	D
	E	

Assume: $\gamma = 1$, $\alpha = 1/2$

Observed Transitions

B, east, C, -2

	0	
0	0	8
	0	

C, east, D, -2

	0	
-1	0	8
	0	

	0	
-1	3	8
	0	

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

Exponential Moving Average

- The current estimate is a weighted average of the previous estimate and the new sample:

$$\bar{x}_n = (1 - \alpha) \cdot \bar{x}_{n-1} + \alpha \cdot x_n$$



$$\bar{x}_n = \frac{x_n + (1 - \alpha) \cdot x_{n-1} + (1 - \alpha)^2 \cdot x_{n-2} + \dots}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots}$$

- Recent samples have higher weights;
- Older samples fade exponentially.
- TD update is an exponential moving average of value estimates.
 - Fixed α : estimates may fluctuate;
 - Decaying α : estimates stabilize over time.

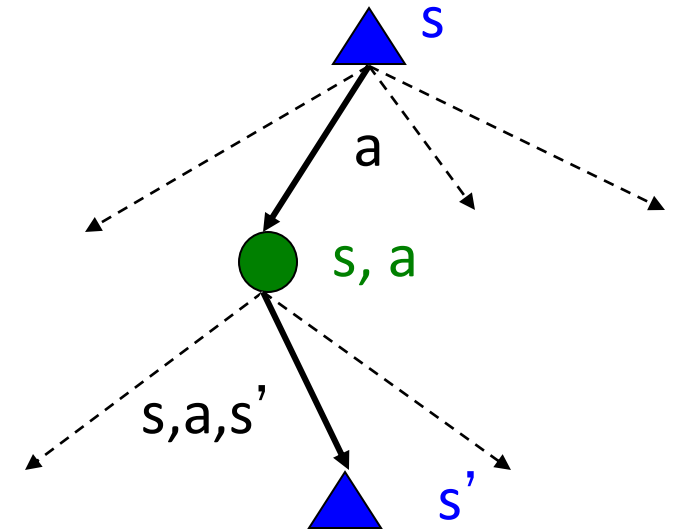
Problems with TD Learning

- TD learning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages.
- However, if we want to turn values into a (new) policy, we're sunk:

$$\pi(s) = \arg \max_a Q(s, a)$$

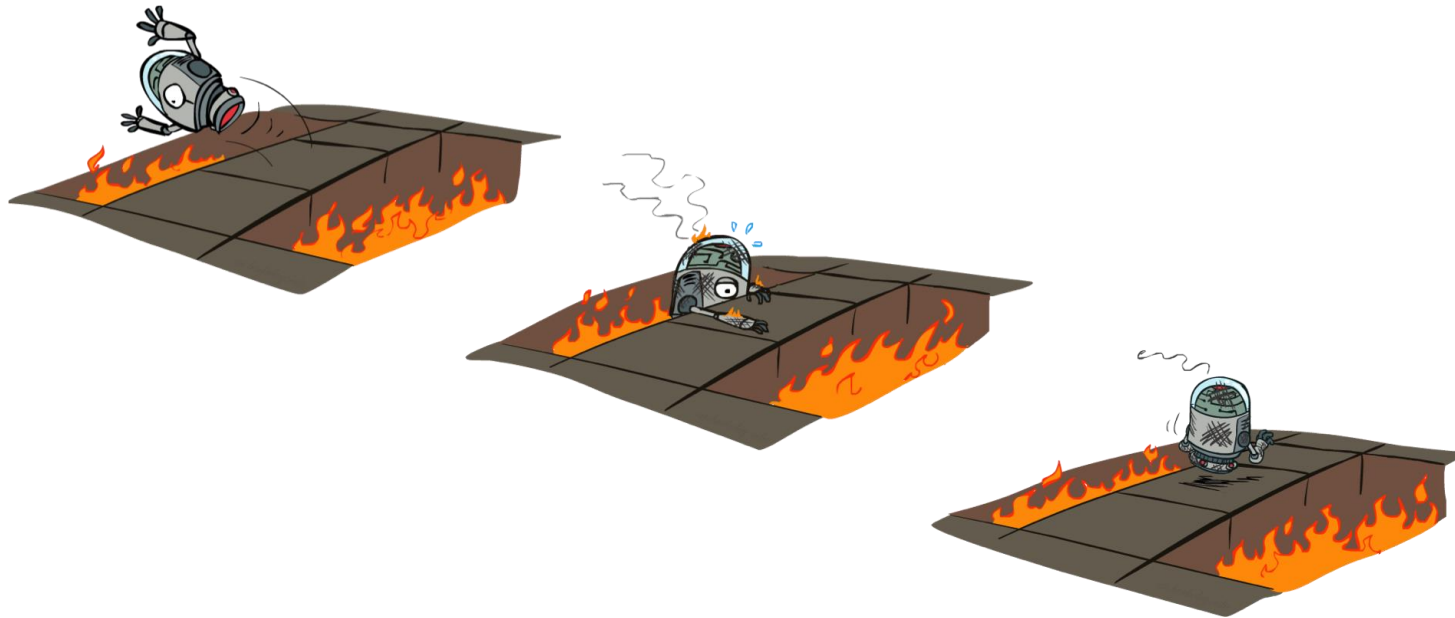
$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

- Idea: learn Q-values, not plain values.
 - Makes action selection model-free too!



Active Reinforcement Learning

- A passive agent follows a fixed policy to determine its actions.
 - It learns from the environment without trying to improve its behavior.
- An active agent not only learns from experience, but also actively updates its policy to make better decisions over time.



Active Reinforcement Learning

- Full reinforcement learning: optimal policies (like value iteration).
 - You don't know the transitions $T(s, a, s')$;
 - You don't know the rewards $R(s, a, s')$;
 - You choose the actions now;
 - Goal: learn the optimal policy / values.
- In this case:
 - Learner makes choices!
 - Fundamental tradeoff: exploration vs. exploitation
 - This is NOT offline planning! You actually take actions in the world and find out what happens...

Recall: (Q-)Value Iteration

- Value iteration: find successive (depth-limited) values.

- Start with $V_0(s) = 0$;
- Given V_k , calculate the depth $k + 1$ values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Q-values are more useful, so compute them instead.

- Start with $Q_0(s, a) = 0$;
- Given Q_k , calculate the depth $k + 1$ Q-values for all Q-states:

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')]$$

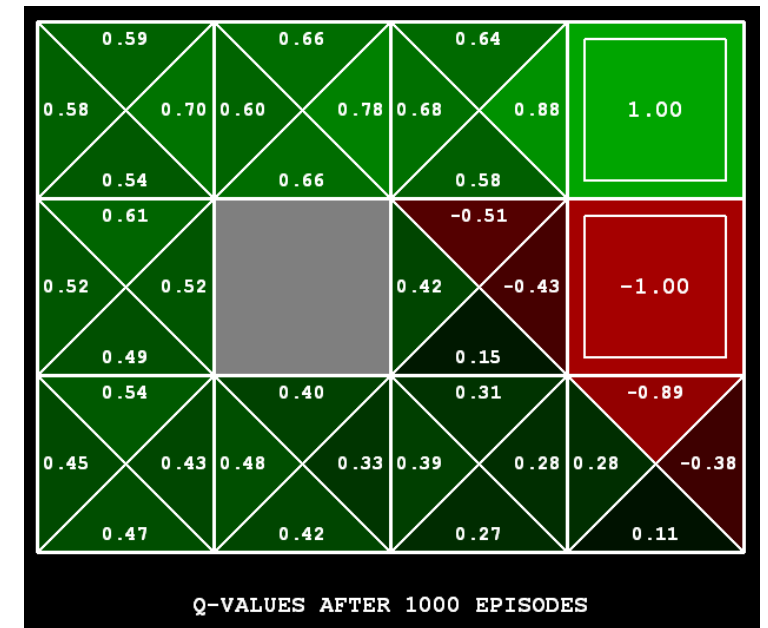
Q-Learning

- Q-Learning: sample-based Q-value iteration.

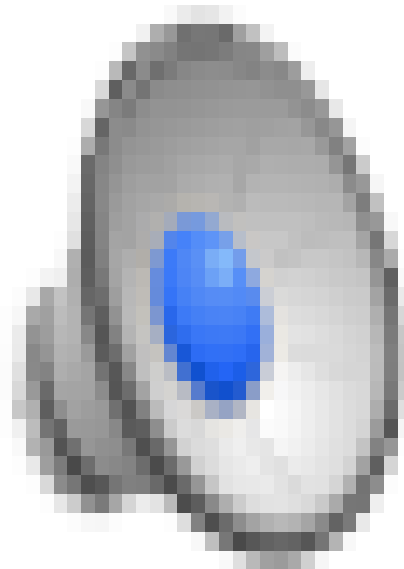
$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right]$$

- Learn $Q(s, a)$ values:

- Receive a sample (s, a, s', r) ;
- Consider old estimate: $Q(s, a)$
- Consider new sample estimate:
 $sample = R(s, a, s') + \gamma \max_{a'} Q(s', a')$
- Incorporate the new estimate into a running average:
 $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + (\alpha) [sample]$



Video of Demo Q-Learning Auto Cliff Grid



Q-Learning Properties

- Q-learning converges to the optimal policy, even if the agent behaves suboptimally.
 - Update rule is based on the Bellman Optimality Equation;
 - Learns action values $Q(s, a)$ through sample transitions;
 - Requires sufficient exploration to discover optimal strategies;
 - Learning rate α should decay over time to ensure convergence.
- Q-learning is called off-policy learning.
 - Learns the optimal policy independently of the agent's behavior;
 - Decouples the behavior policy from the target policy.

Exploration vs. Exploitation

- In reinforcement learning, striking a balance between exploration and exploitation is essential for discovering an optimal policy.
- Exploration:
 - Try unfamiliar actions to uncover their long-term effects;
 - Gather new information about the environment;
 - Enables discovery of better strategies.
- Exploitation:
 - Use current knowledge to select the best-known action;
 - Maximize immediate rewards;
 - Leverages what the agent already knows.

Exploration Strategy

- Simplest: Random actions (ϵ -greedy).
 - With small probability ϵ , act randomly;
 - With large probability $1 - \epsilon$, act on current policy.
- Problems with random actions:
 - Eventually explores the space, but continues to act randomly even after learning stabilizes;
 - Leads to inefficient behavior in the long run
- Exploration decay: Gradually reduce ϵ over time.
 - Shifts focus from exploration to exploitation;
 - Helps the agent settle into optimal behavior.



Exploration Strategy

- Exploration function: Designed to promote exploration of actions that haven't been tried often.
 - Modifies the value function to reflect both **known rewards** and **uncertainty due to limited experience**.
 - Example:

$$f(Q(s, a), N(s, a)) = Q(s, a) + \frac{c}{N(s, a) + 1}$$

- Combines estimated value $Q(s, a)$ with a bonus for infrequent actions;
- When visit count $N(s, a)$ is small, the function returns a more optimistic utility.

Regular Q-Update: $Q(s, a) \leftarrow_{\alpha} R(s, a, s') + \gamma \max_{a'} Q(s', a')$

Modified Q-Update: $Q(s, a) \leftarrow_{\alpha} R(s, a, s') + \gamma \max_{a'} f(Q(s', a'), N(s', a'))$

Exploration Function

- An alternative approach includes potential functions that estimate the potential for reward based on the state-action space:

$$f(Q(s, a), P(s, a)) = Q(s, a) + \beta \cdot P(s, a)$$

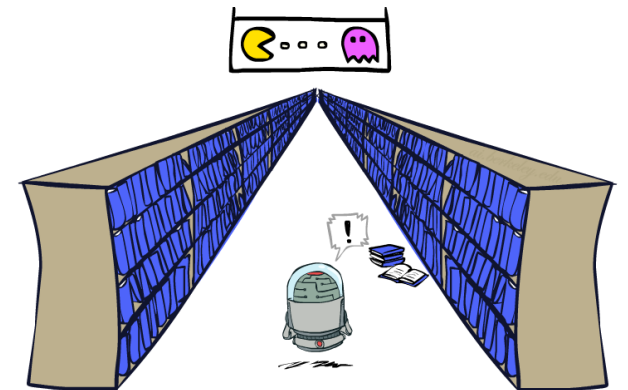
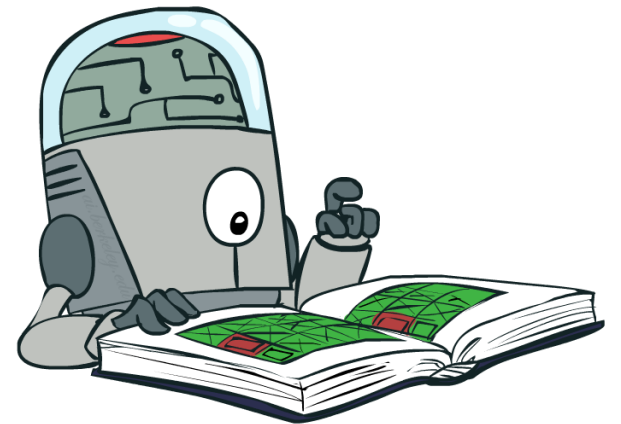
where β is a constant, and $P(s, a)$ represents the potential or uncertainty of the state-action pair.

Regret

- Even if the agent eventually learns the optimal policy, it still makes mistakes during learning.
- **Regret** measures the gap between **rewards actually received during learning** vs. **rewards that would have been received by following the optimal policy** from the start.
- Why minimize regret?
 - Learning to be optimal is not enough;
 - We want to learn optimally with minimal wasted effort and missed rewards.
 - Example: Both random exploration and exploration functions can lead to optimal policies, but random exploration incurs higher regret due to inefficient trial-and-error.

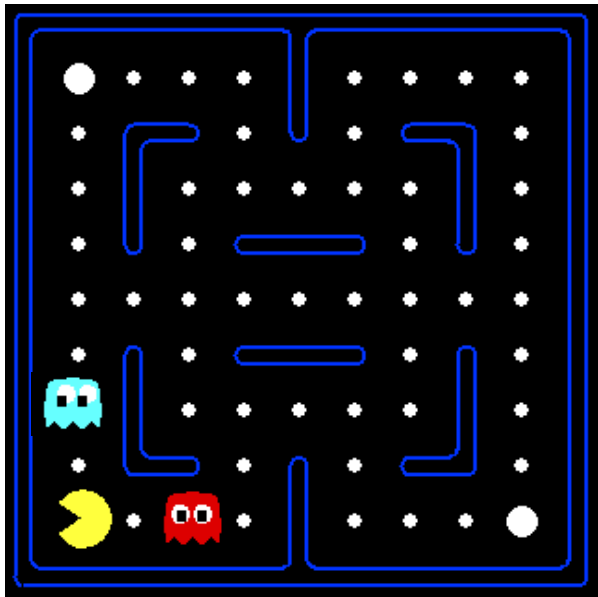
Generalizing Across States

- Basic Q-learning keeps a table of all Q-values.
- In realistic situations, we cannot possibly learn about every single state!
 - Too many states to visit them all in training.
 - Too many states to hold the q-tables in memory.
- Instead, we want to **generalize**:
 - Learn about some small number of training states from experience;
 - Generalize that experience to new, similar situations;
 - This is a fundamental idea in machine learning, and we'll see it over and over again.

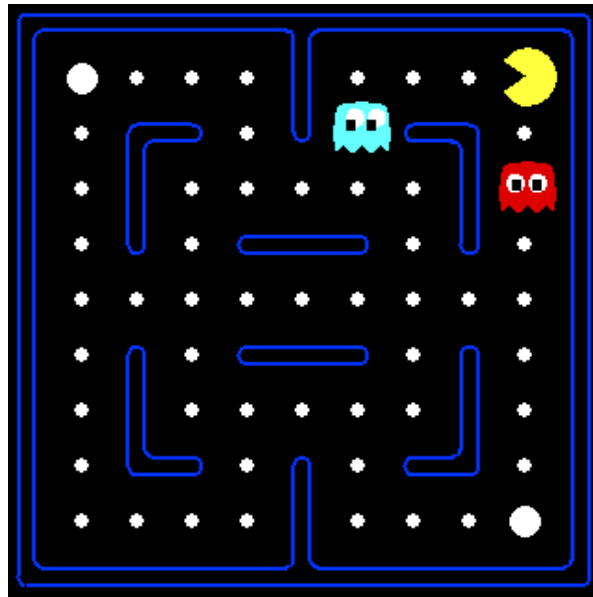


Example: Pacman

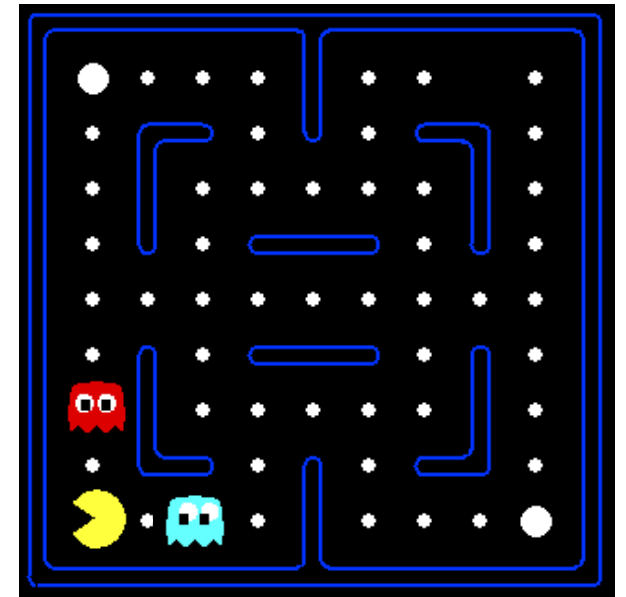
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:

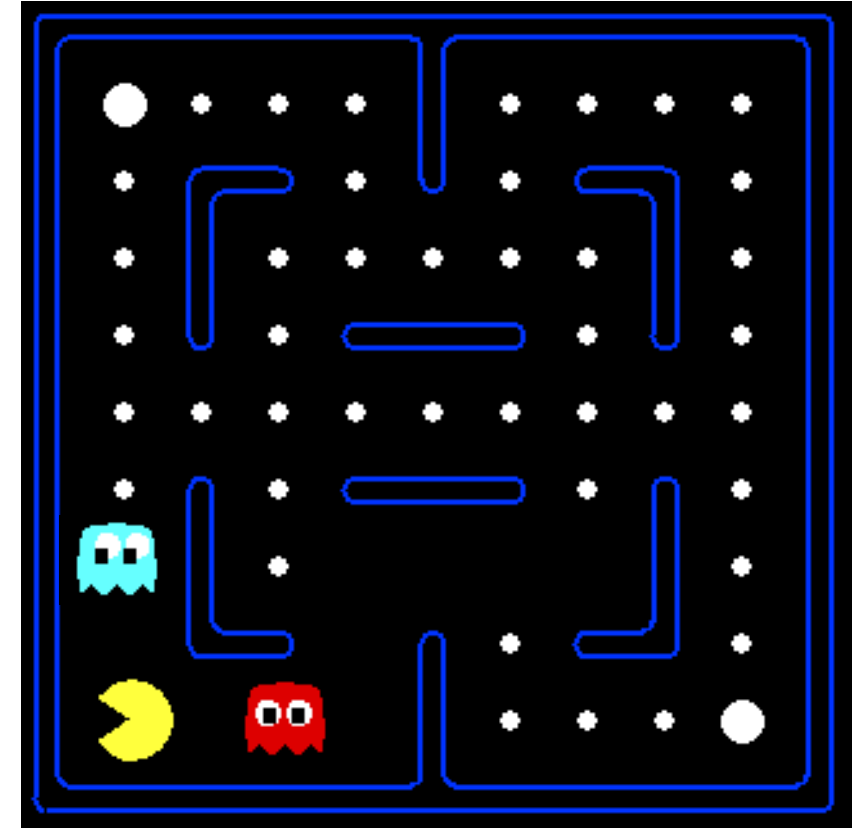


Or even this one!



Feature-Based Representations

- Solution: describe a state using a vector of features
 - Features are functions from states to real numbers (often 0/1) that capture important properties of the state.
 - Example features:
 - Distance to closest ghost;
 - Distance to closest dot;
 - Number of ghosts;
 - $1 / (\text{dist to dot})^2$;
 - Is Pacman in a tunnel? (0/1);
 - etc.
 - Can also describe a Q-state with features (e.g. action moves closer to food).



Evaluation Function

- Using a feature representation, we can write a value function for any state using a few weights:

$$V(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

- The evaluation function provides an approximation for the true utility function or Q-function.
 - Advantage: our experience is summed up in a few powerful numbers.
 - Disadvantage: states may share features but actually be very different in value!

Approximating Q-Learning

- Q-learning with linear Q-functions:

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

$$\text{transition} = (s, a, r, s') \quad \text{difference} = \left[r + \gamma \max_{a'} Q(s', a') \right] - Q(s, a)$$

Exact Q's $Q(s, a) \leftarrow Q(s, a) + \alpha [\text{difference}]$

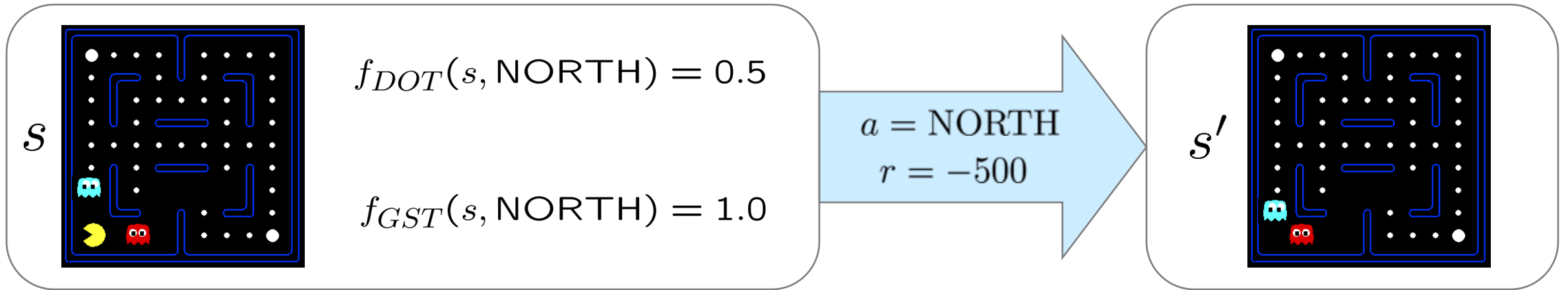
Approximate Q's $w_i \leftarrow w_i + \alpha [\text{difference}] f_i(s, a)$



Adjust weights of active features

Example: Q-Pacman

$$Q(s, a) = 4.0 f_{DOT}(s, a) - 1.0 f_{GST}(s, a)$$



$$Q(s, \text{NORTH}) = +1$$

$$Q(s', \cdot) = 0$$

$$r + \gamma \max_{a'} Q(s', a') = -500 + 0$$

difference = -501

$$w_{DOT} \leftarrow 4.0 + \alpha [-501] 0.5$$
$$w_{GST} \leftarrow -1.0 + \alpha [-501] 1.0$$

$$Q(s, a) = 3.0 f_{DOT}(s, a) - 3.0 f_{GST}(s, a)$$

The Story So Far: MDP and RL

Known MDP: Offline Solution

Goal

Compute V^*, Q^*, π^*

Evaluate a fixed policy π

Technique

Value / policy iteration

Policy evaluation

Unknown MDP: Model-Based

Goal

Compute V^*, Q^*, π^*

Evaluate a fixed policy π

Technique

VI/PI on approx. MDP

PE on approx. MDP

Unknown MDP: Model-Free

Goal

Compute V^*, Q^*, π^*

Evaluate a fixed policy π

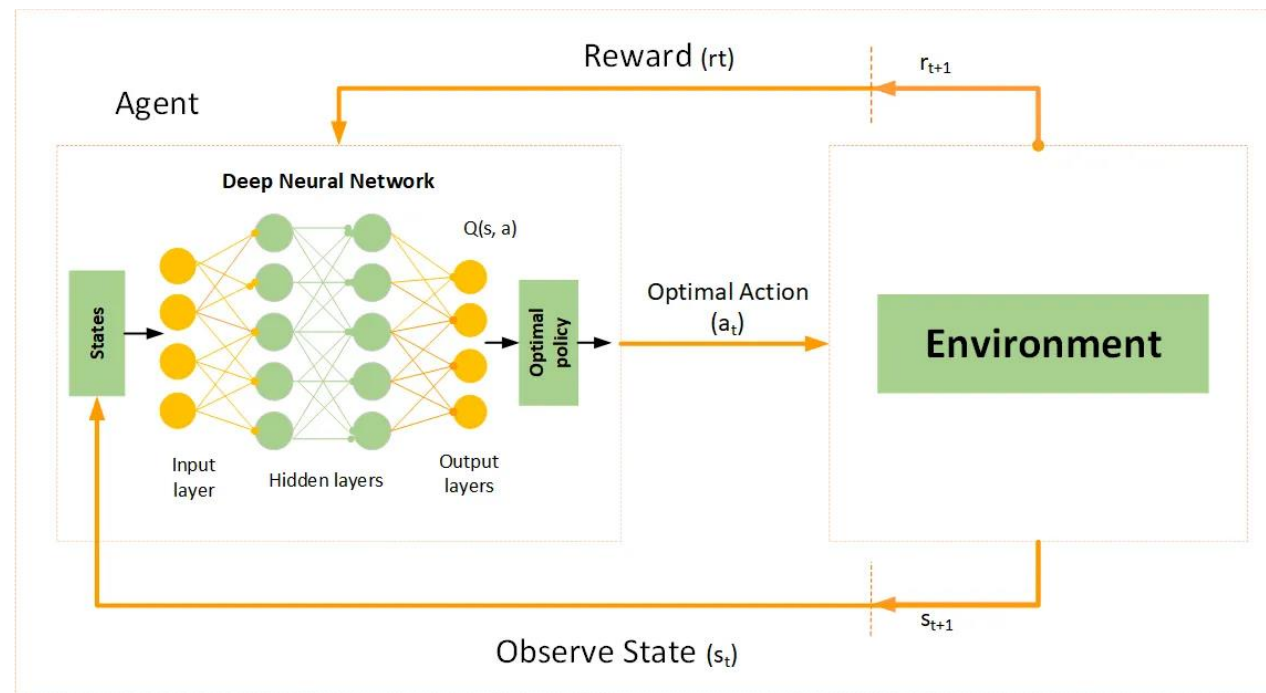
Technique

Q-learning

Value Learning

Deep Q-Network (DQN)

- DQN enhances Q-learning by leveraging deep neural networks to approximate Q-values, enabling effective learning in high-dimensional environments like raw image inputs.



Policy Search

- Value-based methods:
 - Focus on learning value functions $V(s)$ or $Q(s, a)$;
 - Policy is derived by selecting actions that maximize learned values;
 - Example: Q-learning.
- Policy-based methods:
 - Directly parameterize and optimize the policy;
 - No explicit value function is learned;
 - Example: REINFORCE, Actor-Critic.

REINFORCE Algorithm

- REINFORCE is a Monte Carlo policy gradient algorithm.
 - It learns by running full episodes and adjusting the policy based on the total rewards received.
 - If an action led to a good outcome, increase the chance of taking it again.
 - If it led to a bad outcome, reduce that chance.

REINFORCE Algorithm

1. Initialize policy parameters θ :
 - These are the weights of your policy network $\pi_{\theta}(a \mid s)$, which tells you how likely you are to take action a in state s .
2. Generate an episode $(s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T)$ until a terminal state is reached.
3. For each time step t in the episode, compute the total future reward G_t from that time onward:

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

- γ is discounting factor.

REINFORCE Algorithm

4. Estimate the policy gradient:

- Use the **log-likelihood trick** to compute how to adjust the policy:

$$\nabla_{\theta} J(\theta) \approx \sum_{t=1}^T \nabla_{\theta} \log \pi(a_t | s_t; \theta) G_t$$

- This tells you how to change θ to make good actions more likely.

5. Update the policy parameters

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

6. Repeat the process for a number of episodes/iterations until convergence.

Actor-Critic Method

- The actor-critic method combines value function approximation with policy optimization.
 - Actor: This represents the policy $\pi(a \mid s; \theta)$;
 - Critic: This estimates the value function $V(s; w)$ or the action-value function $Q(s, a; w)$
- For each step in an episode:
 - Compute advantage $A_t = G_t - V(s_t; w)$;
 - Update critic parameters using Temporal Difference (TD) error: $w \leftarrow w + \beta \delta_t$;
 - Update actor parameters using policy gradient: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t \mid s_t; \theta) A_t$;

$$\text{difference} = \left[r + \gamma \max_{a'} Q(s', a') \right] - Q(s, a)$$

$$w_i \leftarrow w_i + \alpha [\text{difference}] f_i(s, a)$$

More on Advanced Methods

- Trust Region Policy Optimization (TRPO)
 - Ensures stable learning by constraining policy updates within a trust region using KL divergence.
- Proximal Policy Optimization (PPO)
 - Approximates TRPO with a clipped objective, balancing performance and simplicity for practical use.
- Group Relative Policy Optimization (GRPO)
 - Optimizes policies using contrastive learning with synthetic reference samples and calibrated rewards, enabling stable updates under verifiable or sparse feedback.

PPO for RL Optimization

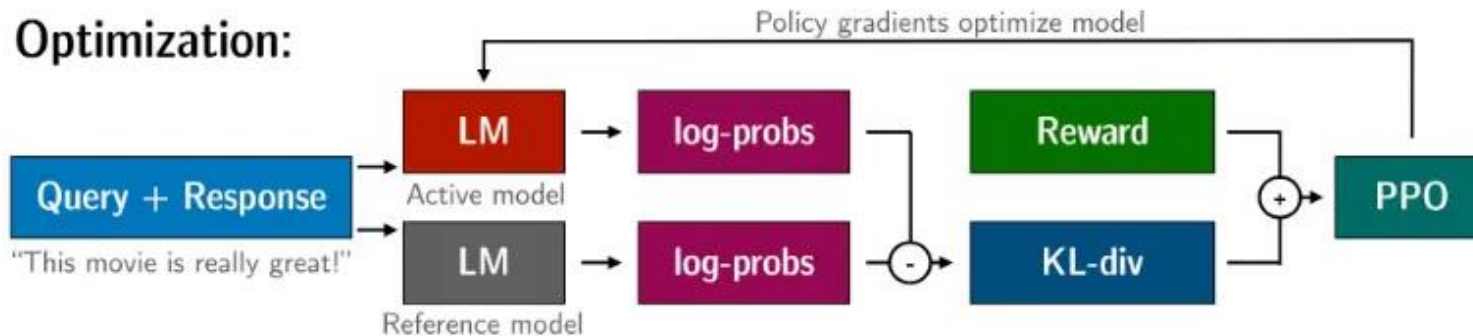
Rollout:



Evaluation:



Optimization:





Summary

- Reinforcement Learning (RL): A framework for training agents to make sequences of decisions by maximizing cumulative reward.
 - Agent: Learner/decision-maker.
 - Environment: Where the agent learns and acts.
 - State (s): Current situation of the agent.
 - Action (a): Choices available to the agent.
 - Reward (R): Feedback from the environment.
 - Policy (π): Strategy mapping states to actions.
 - Value Function (V): Expected long-term return.

Thank You!